The Tree–Walk Hashing Kernel

Reference algorithm and optimized implementation

Part I states the algorithm exactly as the grader's reference implements it. **Part II** explains how the optimized kernel runs the same computation in **1,053** simulated cycles instead of the shipped baseline's **147,734** (a 140× reduction), broken into three independent concerns: *algorithmic* changes (what is computed), *slot allocation* (how operations are packed onto the machine's parallel engines each cycle), and *memory allocation* (how the 1,536-word scratch space is used).

Part I — The reference (naive) algorithm

1 Overview

The workload is a fixed-length, data-dependent walk over a static binary tree, run independently for a batch of walkers. Each walker repeatedly reads the value at its current tree node, mixes it into its running state with a 32-bit hash, and uses one bit of the new state to choose which child to descend to. All arithmetic is over the ring $\mathbb{Z}_{2^{32}}$: every operation is reduced modulo 2^{32}.

2 Data

Tree. An implicit perfect binary tree of height H, stored as a flat array $T[0 \ldots N-1]$ of $N = 2^{H+1} - 1$ node values. Node x has children $2x+1$ (left) and $2x+2$ (right); the root is index 0. Each $T[x]$ is a fixed value in $[0,\ 2^{30}-1]$.

Walkers. A batch of B walkers, each with an index $I[i] \in \{0, \ldots, N-1\}$ and a state value $V[i] \in \mathbb{Z}_{2^{32}}$. Initially every walker sits at the root, $I[i] = 0$, and $V[i]$ is a fixed value in $[0,\ 2^{30}-1]$. The walk runs for R rounds.

3 The hash function

myhash : $\mathbb{Z}_{2^{32}} \to \mathbb{Z}_{2^{32}}$ is applied in six stages. Writing a for the value entering a stage, each stage recomputes

$$a \ \leftarrow\ \big((a \,\square_1\, v_1) \,\square_2\, (a \,\square_3\, v_3)\big) \mod 2^{32},$$

where the two half-expressions are each reduced modulo 2^{32} before being combined by $\square_2$. The operators $\square$ are add ($+$), xor ($\oplus$), left shift ($\ll$), or right shift ($\gg$). The six stages use:

stage	$\square_1$	v_1	$\square_2$	$\square_3$	v_3
1	$+$	`0x7ED55D16`	$+$	$\ll$	12
2	$\oplus$	`0xC761C23C`	$\oplus$	$\gg$	19
3	$+$	`0x165667B1`	$+$	$\ll$	5
4	$+$	`0xD3A2646C`	$\oplus$	$\ll$	9
5	$+$	`0xFD7046C5`	$+$	$\ll$	3
6	$\oplus$	`0xB55A4F09`	$\oplus$	$\gg$	16

Explicitly, with C_k the constant of stage k (both halves read the same input a; the update is not sequential within a stage):

$$a \leftarrow (a + C_1) + (a \ll 12), \qquad a \leftarrow (a \oplus C_2) \oplus (a \gg 19), \qquad a \leftarrow (a + C_3) + (a \ll 5),$$
$$a \leftarrow (a + C_4) \oplus (a \ll 9), \qquad a \leftarrow (a + C_5) + (a \ll 3), \qquad a \leftarrow (a \oplus C_6) \oplus (a \gg 16).$$

4 The per-round update and full algorithm

For a single walker with index x, value v, one round reads $n = T[x]$, sets $v \leftarrow \text{myhash}(v \oplus n)$, descends $x \leftarrow 2x + 1 + (v \bmod 2)$ (left child if v even, right if odd), and wraps $x \leftarrow 0$ if $x \geq N$. The reference iterates rounds outside, walkers inside:

```
for h ← 1 to R:
    for i ← 1 to B:
        n ← T[ I[i] ]
        V[i] ← myhash(V[i] ⊕ n)
        I[i] ← 2 I[i] + 1 + (V[i] mod 2)
        if   I[i] ≥ N   then   I[i] ← 0
```

The graded output is the final value array $V[0 \ldots B-1]$; the indices are internal bookkeeping. The benchmark instance is $H = 10$ ($N = 2047$), $B = 256$, $R = 16$ — $B \cdot R = 4096$ hash evaluations. The tree has 11 levels; a walker at level ℓ moves to level $\ell+1$ each round (wrapping from the bottom), so over 16 rounds it visits levels $0, 1, \ldots, 10, 0, 1, 2, 3, 4$.

Part II — The optimized implementation

5 The target machine

The kernel is scored on a single-core VLIW/SIMD simulator. Each cycle issues one *bundle* of *slots* spread across six engines, up to a per-engine limit; the cycle count is the number of non-trivial bundles. Vector ops act on `VLEN` = 8 lanes at once. Scratch is a flat 1,536-word array that serves as *both* the register file and working memory — there is no separate register space; `mem` is a large separate array reached only through the load/store engines.

engine	slots/cyc	what it does
`alu`	12	scalar integer ops (one lane each)
`valu`	6	vector ops, 8 lanes/slot; **the only engine with fused** `multiply_add` ($a \cdot b + c$)
`load`	2	`mem` read: scalar gather `mem[scratch[addr]]`, or `vload` of 8 contiguous words
`store`	2	`mem` write (scalar or 8-wide `vstore`)
`flow`	1	`select`/`vselect` (8-lane mux) and `add_imm` only — no arithmetic
`debug`	64	correctness compares; free (never counted in cycles)

We measure work in two units: a *slot* is one engine issue-slot in one cycle; a *lane-op* is one scalar-equivalent operation (a `valu` slot does 8 lane-ops, an `alu` slot does 1). The peak arithmetic throughput is $12 + 6 \cdot 8 = 60$ lane-ops/cycle across `alu` + `valu`.

6 Result and where the cycles go

cycles	milestone
147,734	shipped scalar baseline (1 op/bundle)
12,414	vectorize: 8 walkers per `valu` op
4,990	software-pipeline the phases
2,148	fused hash + carry gather address + gather-under-compute
1,572	tournament routing of tree levels 1–3 (fewer gathers)
1,140	group skewing (overlap compute- and load-heavy blocks)
1,053	**current** (constant commuting, engine racing, drain/ramp fixes)

At 1,053 cycles the two arithmetic engines are jointly saturated and the schedule is within 1.7% of the binding resource floor:

engine	slots used	utilization	lane-ops
`valu`	6,209	98.3% (floor $6209/6 = 1035$ cyc)	—
`alu`	12,169	96.3%	—
`load`	1,924	91.4% (1,875 gathers)	—
`flow`	637	60.5%	—
`store`	38	1.8%	—
work by purpose (lane-ops, % of total 68,801):			
Hash			46,656 (67.8%)
Routing			12,409 (18.0%)
Index			9,144 (13.3%)
Setup/Store			592 (0.9%)

The binding engine is `valu` (its fused multiply-add has no scalar substitute), so the optimizations below are ultimately judged by how many `valu` slots they remove or how well they keep all six engines busy in the same cycle.

7 Algorithmic improvements

These change *what* is computed — the operation count and data-flow structure — while leaving the graded output bit-identical.

1. Exploit walker independence (vectorize). Within a round the $B = 256$ walkers are mutually independent, so they pack 8-to-a-lane onto `valu`: 256 walkers become 32 *groups* of 8. Only the 16 rounds are sequential. This alone is the 147k → 12k step and is the foundation everything else builds on.

2. Hash fusion. Three of the six stages $(1, 3, 5)$ have the affine shape $(a + C) + (a \ll s) = a\,(1 + 2^s) + C$, which is exactly one `multiply_add`. Better, stages 2 and 3 *compose* across their boundary: both branches of stage 3 are affine in stage 2's input, so the pair collapses to two multiply-adds plus one xor rather than four ops. The 6-stage hash reduces from a naive ∼18 operations to **11 mixing operations** per evaluation. An exhaustive machine search (adjacent-segment plus meet-in-the-middle, $\sim 2.4 \times 10^{12}$ candidates with constants solved over all 2^{32}) found no shorter form at any cut point; 11 is very likely minimal for this composition, though not proven globally.

3. Commute a constant across the round boundary. Stage 6's constant xor $\oplus C_6$ can be pushed out of the per-round hash by pre-transforming the tree once: if the stored node values carry C_6 already, the fold-in $v \oplus n$ absorbs the elided $\oplus C_6$ from the previous round. This removes one operation per evaluation on 9 of the 16 rounds. Parity flips by $C_6 \bmod 2$ but is corrected for free by reversing the routing tables and adjusting compile-time offset constants.

4. Tournament routing (the load→arithmetic trade). The naive gather $T[I[i]]$ is a data-dependent `load`; with one per walker per round that is $4096/2 = 2048$ load-cycles — the original wall. But the *shallow* tree levels are tiny and shared: level d has only 2^d distinct nodes, identical for all walkers. Those levels are pre-broadcast into scratch once, and each walker selects its node with a *tournament* of 2^d-1 multiplexes driven by its accumulated branch-parity bits, running on `valu`/`flow` instead of `load`. Levels 1–4 are served this way (a partial fifth level too); levels 5–10 still gather. This cuts gathers from 4096 to ∼1875 and moves the bottleneck off the 2-wide `load` engine onto the arithmetic engines — the decisive $2148 \to \sim 1400$ structural change. A walker's position within a level is the d-bit number formed by its last d parity bits (oldest = most significant).

5. Position-accumulator index recurrence. Rather than recompute a full tree address each round, a walker on tournament levels carries a compact position $p \leftarrow 2p + b$ (one multiply-add; b is the new parity bit), and only materializes a real gather address $g = 2p+b+(\text{base}+2^L-1)$ at the boundary where it leaves tournament levels for gather levels. The "+1" of $2x+1$ and the tree base pointer fold into compile-time constants. The whole index recurrence costs ∼2.2 lane-ops per evaluation — below the naive 3-op recurrence.

6. Deterministic wraparound. Because every walker advances exactly one level per round in lockstep, the "if $x \geq N$ wrap to 0" branch fires on the same known rounds for all lanes at once. It is compiled away entirely — there is no per-walker compare or select for wraparound; the position state is simply re-seeded from the root on those rounds.

8 Slot allocation

These do not change the computation; they decide *which engine and which cycle* each operation issues on, to keep the six parallel engines full.

The list scheduler. Operations are emitted as an unordered pool and placed greedily at the earliest cycle with (a) all inputs ready and (b) a free slot on a capable engine, tracking read/write hazards per scratch address. This replaced a fixed "wave of six" hand-schedule and is what lets the following placement tricks compose.

Multi-encoding engine racing. Many operations have more than one legal encoding, and the scheduler places each on whichever engine retires it soonest:

- An 8-wide elementwise vector op costs *one* `valu` slot or *eight* scalar `alu` slots. When `valu` is backed up and `alu` has idle lanes, the split wins — "`alu` offload" raising effective throughput toward 60 lane-ops/cycle.
- A tournament fold is either a `valu` multiply-add or a `flow vselect` (the parity is a raw 0/1 condition). Each fold races the two; ∼54% move to the otherwise-idle `flow` engine, but only where they do not serialize on its single slot.
- The index multiply-by-2 recurrence spells either as one `valu` multiply-add or as `shift`+`add` across 8 `alu` lanes, chosen per site.

A hard, unconditional version of any of these *loses* (measured): the `flow` engine is 1-wide and its idle time is anti-correlated with when folds are ready, so blind offloading serializes. Only the per-site race wins.

Cut the operation, not just its placement. Tournament select conditions are taken directly from the raw parity vectors (which already exist for the index update, and ride an otherwise-dead buffer) instead of being re-extracted with mask-and-shift ops — eliminating those extractions from `alu`/`valu` at zero scratch cost.

Group skewing (software pipeline across blocks). The 32 groups are emitted in 4 blocks staggered 3 rounds apart. A round that is compute-heavy (a tournament level, no gathers) for one block then overlaps a round that is load-heavy (a deep gather level) for another, so `load` and `valu` stay busy in the same cycles instead of taking turns. This was the single largest scheduling win ($1418 \to 1140$).

Setup and drain scheduling. The startup ramp and final-round drain are where engines idle. Setup constants are computed on the idle `alu` (e.g. one constant is another shifted) instead of consuming `load` slots; the 32 base-address adds are moved off the 1-wide `flow` engine into parallel `alu` chains; disjoint final stores are paired to use both `store` slots; and the last round's tournament fold is reordered so the last-arriving parity bit selects last, shortening the drain's critical path.

9 Memory allocation

Scratch is only 1,536 words and doubles as the register file, so every live value competes for it. The kernel runs at **1,535/1,536** words — essentially full — which is only possible because of the following.

Two tiers. `alu`/`valu`/`flow` operate directly on scratch; the large `mem` array is reachable only via load/store. Values that must persist live in scratch; anything that can be recomputed or streamed from `mem` need not.

Persistent state vs. reused temp pools. Each of the 32 groups keeps exactly three live 8-word vectors across the whole run: its hash state, its node-value buffer, and its position/address accumulator ($32 \times 3 \times 8 = 768$ words). Everything else — hash intermediates, tournament condition vectors — comes from a small *shared pool* of temporaries (sizes $(16, 4)$) that is reused group-to-group and wave-to-wave, rather than allocated per group. Pool sizing is a direct scratch-vs-parallelism knob: too small serializes on write-after-write hazards, too large overflows.

Constant coalescing. The register-pressure analysis initially read a peak of $\sim$2300 live words — but $\sim$2048 of those were redundant per-walker copies of a handful of *compile-time constants* (gather-base offsets, the reset position). Coalescing identical constants to one shared broadcast word each — what a real vectorized kernel does — collapses the genuine working set to $\sim$780 words, which is what makes the whole design fit without spilling.

Shared tournament tables and in-`mem` priming. The tree is static, so each served level's value/difference tables are built into scratch *once* at setup and shared by all groups. For the constant-commuting transform, the deep-level tree values are pre-xored with C_6 directly in `mem` at setup (using the near-idle `store` engine during the ramp), so no per-round scratch or work is spent maintaining the transformed domain.

Dead-register mining. On the final round, 52 vectors are provably dead (the position accumulators of groups that no longer gather, and the node-value buffers of already-finished blocks). The drain-shortening optimization reuses exactly those dead words to hold its extra tables, funding a genuine improvement with *zero* net scratch — the only reason it fits at 1,535/1,536.

10 Lower bounds and the remaining gap

The hash must be evaluated 4096 times; that work alone, on the 60-lane-op/cyc `alu`+`valu` pair, is a hard floor of ~ 780 cycles. Adding the index recurrence gives ~ 930; adding the routing folds that cannot fit on the 1-wide `flow` engine gives the realized `valu` floor of ~ 1035, and the kernel runs at 1,053 — 1.7% above it. The gap from 930 to 1035 is an irreducible "routing tax" imposed by the ISA: `flow` has only one slot per cycle, so the selects that would otherwise offload from the saturated arithmetic engines serialize instead. Going below 1,000 would require removing $\sim$215 `valu` slots of real arithmetic — i.e. a hash shorter than 11 operations (searched to 2.4×10^{12} candidates without a hit) or a structurally cheaper routing scheme — not better scheduling, which is already essentially optimal.